

CampusLink

A Goal-Matching & Resource-Sharing Platform for College Campuses

Project Proposal for UCS503

Submitted to: Ma'am Anushka

Thapar Institute of Engineering and Technology

Govind, CSED — Roll No: 1024030968, Email: gpodder_be24@thapar.edu

Ashish, CSED — Roll No: 1024030096, Email: Kashish_be24@thapar.edu

[Teammate Name], CSED — Roll No: [roll no], Email: [email]

August 2026

1. Higher-order goal

The broader aim of this project is to make it easier for students on a large campus to find people to do things with, be it a spontaneous plan, a recurring need like a gym partner, or something as simple as sharing class notes. Most campuses already have this problem solved informally through WhatsApp groups and friend circles, but that only works if you already know the right people. CampusLink tries to close that gap by surfacing strangers who want the same thing at the same time, and give the interaction enough structure that it feels safe to do with someone you don't know.

2. Problem Statement

On a residential campus with thousands of students, a lot of everyday coordination still happens through fragmented, informal channels — batch WhatsApp groups, hostel groups, word of mouth. This has a few recurring pain points:

- **Limited reach:** if you don't personally know someone who's free for a gym session or going the same direction for a cab, you simply don't do it, even though statistically someone on campus probably wants the exact same thing.
- **No structure for stranger interaction:** there isn't really a safe, trust-building layer to interact with someone you don't know yet — this matters more for things like late-night rides or one-on-one meetups.
- **Notes/resource sharing is inefficient:** asking around in 5 different groups for last semester's notes, and getting no response, or getting it 2 days after the exam is not that uncommon.
- **No accountability:** people flake on plans all the time and there's no record of it, so the next person has no way to gauge whether someone is reliable before committing time to them.

3. Proposed Solution

3.1 Overview

CampusLink is a companion-matching platform built around a simple loop: a student posts a **goal** (an activity that needs another person), it appears on the public campus feed, interested students **request** to join, and once enough people are in, a small **lobby** forms and the group takes it from there.

The same post → request → fulfil loop is reused for a second, smaller use case — academic resource sharing. A student posts that they need notes for a subject, and whoever has them responds privately in a 1-on-1 chat rather than the public lobby (since this doesn't really need a "group").

3.2 Core Workflow

- **Post** — student creates a goal with category, description, date/time and headcount needed.
- **Discover** — goal shows up on the public feed, filterable by category and date.
- **Request** — interested students request to join; poster approves manually, or auto-accept if the poster set that rule.
- **Coordinate & Close** — group chats in-app, meets up, both sides confirm it happened, and leave feedback.

3.3 Key Features

Signup & Core Posting (MVP)

- College email verification at signup, so the platform stays restricted to actual students
- Post a goal with category, description, date/time, headcount
- Public feed, filterable by category and date
- Request-to-join flow, with manual approval or auto-accept
- Basic profile — name, year, branch, past goals completed
- In-app chat, opens once a lobby is formed
- Separate Notes/Resource Request category, where responders reach out via a private chat instead of a public lobby

Trust & Safety Layer

This is arguably the more important half of the project, since the platform's whole premise is connecting strangers.

- Report / block a user
- Gender-preference filter — a poster (most commonly relevant for female users, but not restricted to that) can restrict who is allowed to request into a specific goal, e.g. a girl posting about a late-night ride home can choose to filter out male requesters entirely
- **Anonymous posting:** a poster can choose to hide their name and photo on the public feed for a given goal. This is meant for cases where the activity itself reveals something the poster may not want visible to everyone on the feed — e.g. posting "need a cab-share to the airport, flying out at 5am" tells anyone watching the feed exactly where you'll be and when.
- Only the category, description, date/time, headcount, and the poster's reliability score are shown on the feed — identity is not.

- Reliability score stays visible even while anonymous, so requesters can still judge trustworthiness without knowing who the person is.
- The requester's own identity is *not* hidden — a poster still needs to see who is requesting in order to approve them.
- Once a request is accepted into the lobby, the poster's full profile becomes visible to that specific requester (but not to the rest of the feed). Anonymity is basically a pre-acceptance shield, not a permanent one.
- Visible reliability score ("goals completed") on every profile, so flaky users are visible before you commit to joining their lobby
- No-show flagging — if someone doesn't show up, it gets logged and factored into their reliability score

Matching Intelligence

- Suggest goals to a user based on shared interests, branch, or categories they've joined before
- Hostel-block / proximity-based matching, useful for local, low-effort goals
- Priority queueing based on reliability/profile completeness rather than pure first-come-first-served, so genuine users don't lose out to someone who just signed up

Post-Match Lifecycle

- Confirmation step — both sides mark that the meetup actually happened
- Rating/feedback after the goal closes, including a separate rating for note quality in the resource-sharing flow
- Auto-archiving of completed or expired goals so the feed doesn't get cluttered

Nice-to-have, if time permits

- Push notifications on lobby activity
- Streaks/badges for reliable users
- "Trending goals" section

3.4 Operational Constraints

- Should work fine on average student phones/laptops — no heavy client-side processing.
- Duplicate/near-duplicate goal detection is not in scope for this course project; keeping the engineering effort on workflow and the trust layer instead of building matching algorithms from scratch.
- Deployable on standard free-tier hosting so it can actually be demoed live and not just run locally.

4. Solution Approach

This being a software engineering lab project, the focus stays on system design, correctness, and the multi-actor workflow rather than building anything research-grade. The "intelligence" in matching intelligence is intentionally simple — rule-based suggestions using shared attributes (branch, interests, past categories), not an ML model. This keeps the project scoped to something that can be engineered properly

and shown as a working, testable system within the semester, rather than getting stuck trying to build a recommendation engine.

4.1 Architecture

A standard 3-tier web app:

- **Frontend** — React (Vite) + Tailwind for the UI, Zustand/Redux Toolkit for managing feed/lobby/chat state, React Router for navigation between feed, profile, chat, and goal-detail pages.
- **Backend** — Node.js + Express (NestJS is also an option depending on how the team wants to structure it), handling auth, the request/lobby state machine, and the matching/priority logic.
- **Database** — MongoDB (or PostgreSQL, TBD once we finalize the schema — a goal/lobby/request relationship might actually be cleaner relationally).
- **Real-time layer** — Socket.IO for live lobby updates and in-app chat, since polling for chat would be a bad experience.
- **Auth** — JWT + college email OTP verification.
- **File storage** — Cloudinary/Firebase, for notes uploads and profile photos.
- **Deployment** — Vercel for frontend, Render/Railway for backend, so there's a shareable live demo link and not just a local build.

5. Evaluation Criteria

5.1 Primary metric

Lobby fulfilment rate — percentage of posted goals that reach their required headcount within a reasonable window (say, the goal's own date/time, or 48 hours for undated ones). This is the metric that actually reflects whether the core loop works.

5.2 Secondary metrics

- **Time-to-first-request** — how long a goal sits on the feed before someone requests to join. Lower is better, and it's a decent proxy for whether the feed/discovery/filtering is doing its job.
- **No-show rate** — percentage of confirmed lobbies where at least one member doesn't show. This is really the metric that tells us if the trust & safety layer (reliability score, no-show flagging) is having any effect over time.
- **Resource-request response rate** — percentage of notes/resource requests that get at least one private response.
- **User-reported clarity/safety** — a short 1–5 feedback question after using the anonymous or gender-filter features, to check if people actually feel safer, or if it's just a feature nobody trusts enough to use.

5.3 Pilot Validation

We plan on running an internal pilot with 15–20 students within our own batch/hostel block, mixing genuine goal posts with a few seeded ones to make sure there's activity to test the request/lobby flow against, since a cold feed with zero users doesn't really tell us anything about whether the matching or trust layer works.

6. Scalability

- **Theoretical** — frontend and backend are decoupled from the start, the feed query is paginated and indexed on category/date, and the API is stateless so it can scale horizontally if it ever needed to (not that a college project needs to handle campus-wide load, but the design shouldn't actively prevent it).
- **Practical** — runs on standard free/low-cost hosting (Vercel + Render/Railway), uses a managed database, so there's nothing here that needs custom infra to deploy or maintain.

7. Tooling / Engine Availability

Nothing in this stack needs to be built from scratch — all mature, well-documented tools:

- Web framework for REST APIs (Express/NestJS)
- Managed document or relational DB
- Token-based auth (JWT), OTP via email service
- Socket.IO for the real-time layer
- Cloudinary/Firebase for file storage
- Jest + React Testing Library for unit/component tests
- Standard CI (GitHub Actions) for build + test on every push

This lets the team spend the actual effort on the workflow, state management (goal → request → lobby → close), and the trust & safety logic, instead of re-inventing infrastructure.

8. Project Scope and Deliverables

8.1 Iteration 1 — Core MVP (Weeks 1–4)

- College email verification, signup/login
- Basic profile pages
- Post a goal, feed display with category/date filters
- Request-to-join flow, poster approval, lobby state management
- Basic CI: build + lint on every push

8.2 Iteration 2 — Trust, Safety & Real-time (Weeks 5–7)

- Socket.IO in-app chat, per lobby, and the private notes-request chat
- Report/block, gender-preference filter, reliability score
- Anonymous posting mode
- No-show flagging
- Match suggestions, hostel/proximity-based matching, priority queueing
- Confirmation step + ratings/feedback

8.3 Iteration 3 — Polish & Deploy (Week 8)

- Responsive UI pass
- Testing (unit + a couple of integration tests on the core loop)
- Deployment to Vercel/Render, README, short demo video

9. Risks and Mitigations

- **Safety of stranger meetups** — the platform can't fully guarantee physical safety once people leave the app; mitigated (partially) by reliability scores, report/block, and the gender filter, but this is a genuine limitation we're upfront about, not something we claim to fully solve.
- **Cold-start problem** — a feed with no active goals is useless. During the pilot we'll seed some goals ourselves to get the loop moving.
- **Anonymity misuse** — someone could post anonymously to say something inappropriate without accountability. Mitigated by keeping reliability score visible even in anonymous mode, and admins being able to unmask a user's identity in case of a report (not shown to the public, only for moderation).
- **Low adoption within course timeline** — since real student adoption takes longer than 8 weeks, the pilot group will be limited to our own batch/hostel for evaluation purposes.

10. Summary

CampusLink is a goal-posting and lobby-matching platform aimed at solving a genuinely everyday problem on campus — finding people to do things with, beyond your own friend circle, without it feeling unsafe to interact with a stranger. The project is scoped to focus on the core workflow (post → request → lobby → close) and a trust & safety layer (block/report, reliability score, gender filter, anonymous posting) that make stranger-to-stranger interaction on campus feel workable, rather than trying to build a research-heavy matching algorithm within a single semester.